

BANGTORY

공동생활관리 서비스

A팀 (김민영, 민수연, 최현경, 홍수한)

TABLE OF CONTENTS

01

프로젝트 소개

개발 배경 및 동기, 기획 목표,
타겟 사용자, 주요 기능

02

주요 기능 및 UI

주요 기능 설명, UI 소개,
시연영상

03

기술 및 구현과정

기술 선정이유, ERD/디렉토리
구조, API설계 및 구현흐름

04

효과, 한계 및 확장

문제 상황과 해결 방법,
기대효과,확장성,한계점

01-1 BACKGROUND

개발 배경 및 동기

HANA CULTURE



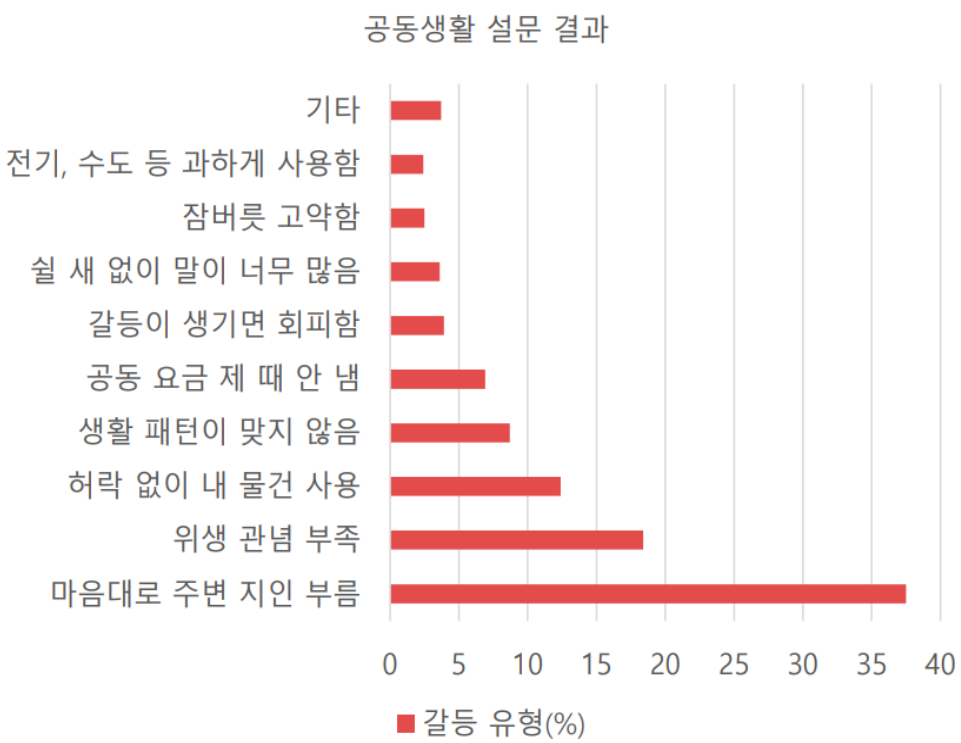
나 혼자 산다!

가성비 주택 코리빙하우스

CO-LIVING

가족이 아닌 사람들이 모여 사는
공동 및 공유 주거 공간

(쉐어하우스, 기숙사)



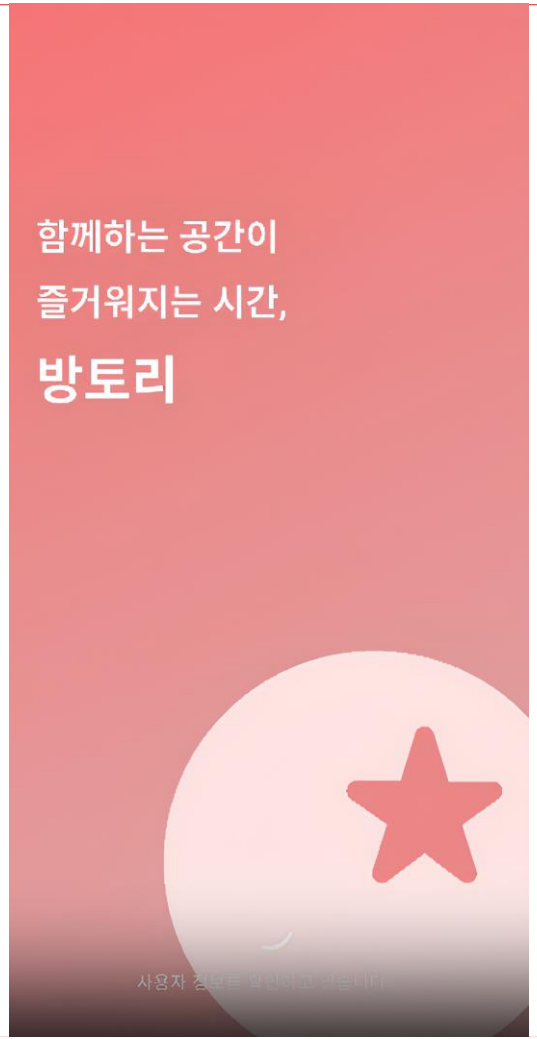
공동 생활 갈등 상황

2021.03.28 ~ 2021.04.01 전국

46,870명 대상 설문조사 결과

01-2 PROJECT GOAL

기획 목표



BANGTORY

"방(Room) " 과 "토리(Story)"의 조합

청소, 예약, 일정 올인원 관리

01-3 TARGET USER

타겟 사용자



TARGET USER

대학교 기숙사나 자취방, 쉐어하우스 등에서 다른 사람과 함께 생활하는 20대 청년 및 대학생

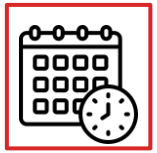
01-4 KEY FEATURES

주요 기능

주요 기능



집안일 관리
(Chore
Management)



일정 예약
(Shared
Schedule
Booking)



룸메이트
초대/추가
(Roommate
Invitation)

주요 기능



홈 화면 사용자 표시
(Roommate Overview)



실시간 반영 및 알림 기능
(Real-time Updates & Notifications)



채팅 기능
(In-App Chat)

02

주요 기능 및 UI

시안 영상
(전체 앱 구성, 핵심 기능 소개, UI 설명)

화면 구성 기준과 특징

랜덤 프로필 기반 UX

앱 전반에 걸쳐 사용자 정보 일관성 있게 표시



사용자 흐름 중심의 전체 설계

앱의 주요 기능 흐름에 맞춰 직관적인 화면 순서 구성

방 중심 구조

모든 기능 '방 단위 ' 로 작동

화면 구성 기준과 특징

직관적인 홈 화면

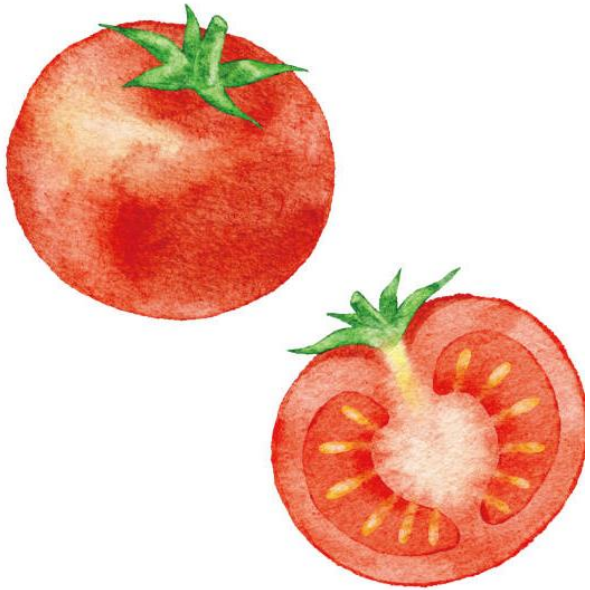
집안일 / 예약
카테고리로 바로 접근

하단바를 통해 핵심
기능 바로 이동

실시간 상호작용

팝업, 새로고침 버튼 등
즉각 반응 가능한 UI

알림 및 메시지



간편한 예약, 등록

가시적인 시간 확인

홈 화면 자동 반영

알림-다른 사용자의
예약 현황 실시간 확인



(76) '방토리' -소프트웨어학과 캡스톤 디자인 최종 시안 영상 - YouTube

03

기술 및 구현과정

기술 선정이유, ERD/디렉토리 구조,
API설계 및 구현흐름

03-1 STACK

기술 선정이유

백엔드 기술 스택



NODE.JS + EXPRESS

- 빠른 개발 속도와 높은 생산성
- 풍부한 npm 생태계



MongoDB + Mongoose

- NoSQL 특성상 스키마 변경이 유연함
- 복잡한 관계형 데이터보다는 문서 기반 데이터 구조에 적합

프론트엔드 기술 스택



Flutter + Dart

- 하나의 코드베이스로 iOS/Android 동시 개발
- Hot Reload를 통한 빠른 개발 사이클

Provider 패턴

- Flutter 공식 권장 상태 관리 솔루션
- 단순하고 직관적인 API

03-2 ERD

ERD 구조

사용자 & 방 관리

User			
ObjectId	_id	PK	
String	name		
String	provider		kakao, google, naver, manual
String	providerId		
DateTime	createdAt		
DateTime	updatedAt		

RoomMember		
ObjectId	_id	PK
ObjectId	userId	FK
ObjectId	roomId	FK
String	nickname	
String	profileImageUrl	
Boolean	isOwner	
DateTime	joinedAt	
DateTime	createdAt	
DateTime	updatedAt	

Room		
ObjectId	_id	PK
String	roomName	
String	address	
ObjectId	ownerId	FK
String	inviteCode	
DateTime	inviteCodeExpiresAt	
DateTime	createdAt	
DateTime	updatedAt	

- **Users ↔ RoomMembers ↔ Rooms**

집안일 관리

Room		
ObjectId	_id	PK
String	roomName	
String	address	
ObjectId	ownerId	FK
String	inviteCode	
DateTime	inviteCodeExpiresAt	
DateTime	createdAt	
DateTime	updatedAt	

ChoreCategory			
ObjectId	_id	PK	
String	name		
String	icon		
String	type		default, custom
ObjectId	room	FK	
ObjectId	createdBy	FK	
DateTime	createdAt		
DateTime	updatedAt		

ChoreSchedule		
ObjectId	_id	PK
ObjectId	room	FK
ObjectId	category	FK
ObjectId	assignedTo	FK
Date	date	
Boolean	isCompleted	
DateTime	completedAt	
ObjectId	createdBy	FK
DateTime	createdAt	
DateTime	updatedAt	

- **ChoreCategories**에서 **ChoreSchedules**로 이어지는 **1:N 관계**

예약 시스템

ReservationCategory			
ObjectId	_id	PK	
String	name		
String	icon		
String	type		default, custom
ObjectId	room	FK	
Boolean	requiresApproval		
Boolean	isVisitor		
ObjectId	createdBy	FK	
DateTime	createdAt		
DateTime	updatedAt		

ReservationSchedule			
ObjectId	_id	PK	
ObjectId	room	FK	
ObjectId	category	FK	
ObjectId	reservedBy	FK	
Number	dayOfWeek		0-6, 일요일=0
Date	specificDate		방문객 예약용
Number	startHour		0-23
Number	endHour		1-24
Date	weekStartDate		주간 예약용
Boolean	isRecurring		
String	status		pending, approved
DateTime	createdAt		
DateTime	updatedAt		

ReservationApproval			
ObjectId	_id	PK	
ObjectId	reservation	FK	
Array	approvedBy		user: ObjectId, approvedAt: Date
Number	totalMembersCount		
Boolean	isFullyApproved		
DateTime	createdAt		
DateTime	updatedAt		

- **ReservationCategories**에서 **ReservationSchedules**로 이어지는 **1:N 관계**
- **ReservationApprovals**라는 **별도 테이블**

소통 & 알림 기능

ChatMessage			
ObjectId	_id	PK	
ObjectId	roomId	FK	
ObjectId	userId	FK	
String	message		
String	messageType		text, system
DateTime	timestamp		
Boolean	isDeleted		
DateTime	createdAt		
DateTime	updatedAt		

Notification			
ObjectId	_id	PK	
ObjectId	userId	FK	
ObjectId	fromUserId	FK	
ObjectId	roomId	FK	
String	type		member_joined, member_left, member_kicked, ownership_transferred, chore_assigned, chore_completed, reservation_created, reservation_approved, visitor_request, category_created, room_updated, invite_code_generated
String	title		
String	message		
JSON	relatedData		
Boolean	isRead		
DateTime	readAt		
DateTime	createdAt		
DateTime	updatedAt		

- **ChatMessages**와 **Notifications** 모두 **방과 사용자를 연결**
- **Notifications**: 모든 활동을 연결하는 알림 허브

주요 설계 원칙

1. **단일 방 참여**: 사용자는 한 번에 하나의 방에만 참여 가능하도록 제한

2. **방장 시스템**: 방 생성자가 자동으로 방장이 되며, 필요시 다른 멤버에게 위임 가능

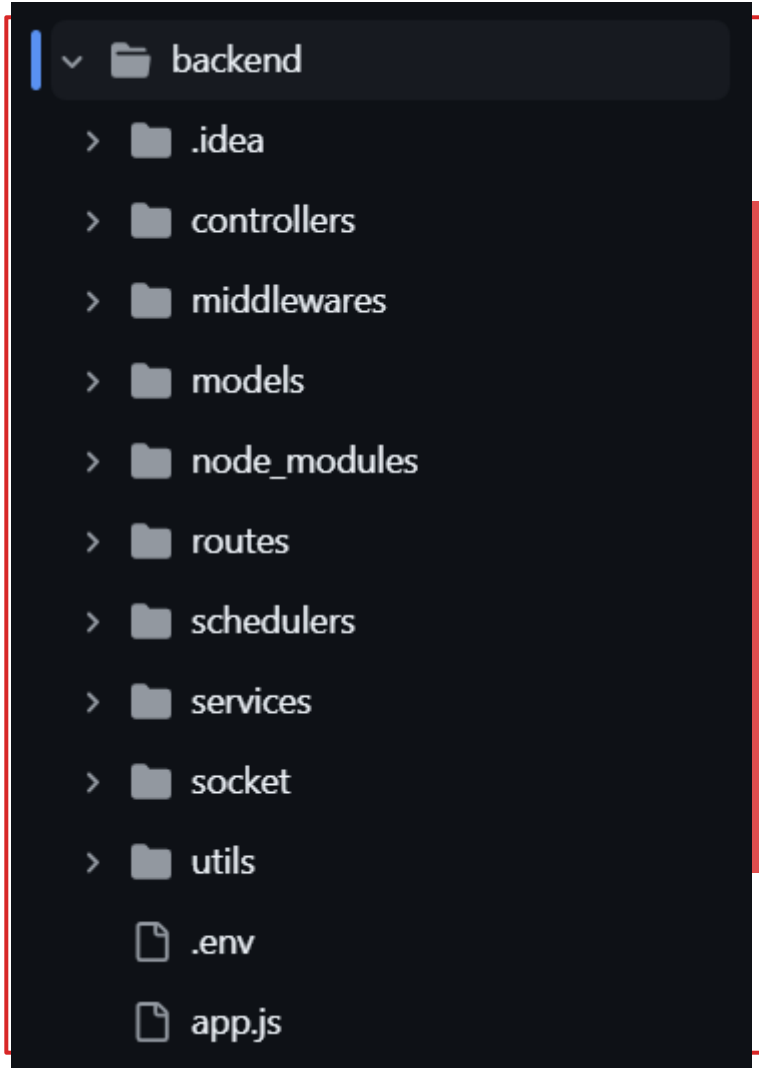
3. **카테고리 기반**: 집안일과 예약 모두 **유연한 카테고리 시스템**을 사용

4. **승인 시스템**: 방문객 예약은 **다른 멤버들의 승인**이 필요한 민주적 시스템

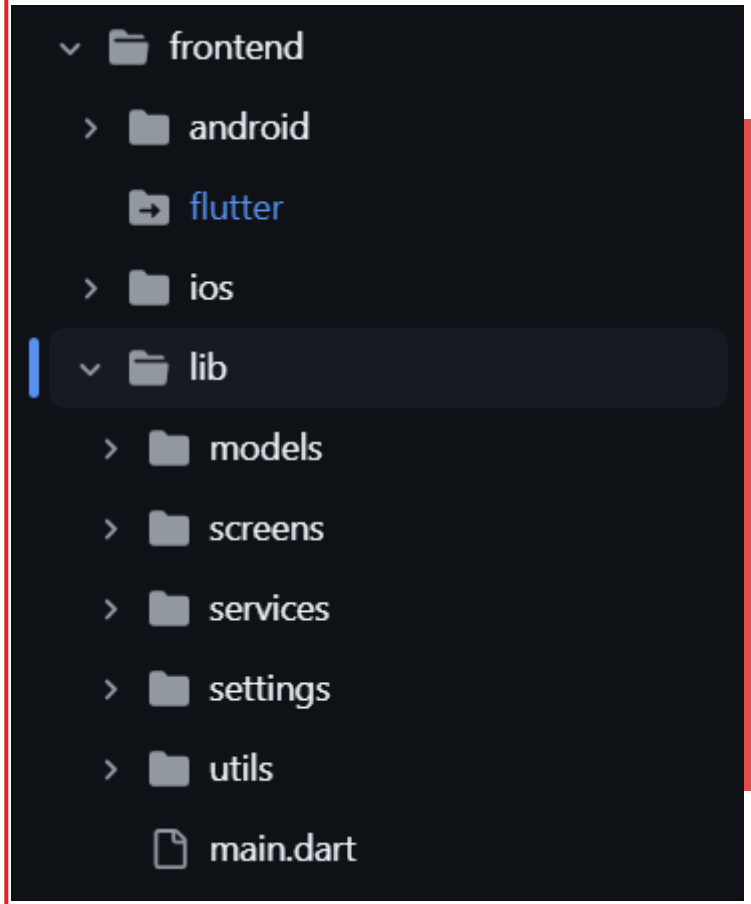
5. **실시간 알림**: 모든 중요한 활동에 대해 **즉시 알림**을 제공

03-3 DIRECTORY

디렉토리 구조



- **MVC 패턴**: Controller-Service-Model로 책임을 명확히 분리
- **Repository 패턴**: Service 계층에서 데이터 접근 로직을 추상화



- **Provider 패턴:** Flutter에서 상태 관리와 의존성 주입

- **Factory 패턴:** 모델 객체 생성 시 fromJson 팩토리 메서드 활용

03-4 API

API설계

주요 API 엔드포인트

사용자 관리

POST	/users	# 사용자 생성
GET	/users/me	# 내 정보 조회
GET	/users/me/profile	# 프로필 정보 조회
PATCH	/users/me/profile	# 프로필 수정

주요 API 엔드포인트

방 관리

```
POST   /rooms                          # 방 생성
POST   /rooms/join                     # 방 참여
GET    /rooms/me                       # 내 방 조회
POST   /rooms/invite                   # 초대코드 생성
PATCH  /rooms/:id                      # 방 정보 수정
DELETE /rooms/:id                      # 방 삭제
DELETE /rooms/leave                    # 방 나가기
GET    /rooms/:id/members              # 방 멤버 목록
PATCH  /rooms/:id/transfer-ownership # 방장 위임
DELETE /rooms/:id/members/:userId # 멤버 내보내기
```

주요 API 엔드포인트

집안일 관리

```
GET      /chores                          # 카테고리 목록 조회
POST     /chores                          # 카테고리 생성
DELETE /chores/:id                        # 카테고리 삭제


GET      /chores/schedules                # 일정 목록 조회
POST     /chores/schedules                # 일정 생성
PATCH    /chores/schedules/:id/complete # 일정 완료
PATCH    /chores/schedules/:id/uncomplete # 완료 해제
DELETE /chores/schedules/:id        # 일정 삭제
```

주요 API 엔드포인트

예약 관리

GET /reservations/categories # 예약 카테고리 목록

POST /reservations/categories # 예약 카테고리 생성

DELETE /reservations/categories/:id # 카테고리 삭제

POST /reservations/schedules # 예약 생성

GET /reservations/weekly-schedules/:roomId # 주간 예약 조회

GET /reservations/visitor-schedules/:roomId # 방문객 예약 조회

GET /reservations/pending-schedules/:roomId # 승인 대기 예약

PATCH /reservations/schedules/:id/approve # 예약 승인

DELETE /reservations/schedules/:id # 예약 삭제

주요 API 엔드포인트

채팅

```
GET     /chat/messages/:roomId     # 채팅 메시지 조회
POST    /chat/messages             # 메시지 저장
DELETE /chat/messages/:id          # 메시지 삭제
```

알림

```
GET     /notifications             # 알림 목록 조회
GET     /notifications/unread-count # 읽지 않은 알림 개수
PATCH   /notifications/:id/read    # 알림 읽음 처리
PATCH   /notifications/read-all    # 모든 알림 읽음 처리
```

03-5 FLOW

구현 흐름

핵심 구현 흐름

1. 인증 흐름

1. 사용자가 이름 입력
2. POST /users로 사용자 생성
3. 응답에서 받은 userId를 SharedPreferences에 저장
4. 이후 모든 API 요청 시 X-User-ID 헤더에 포함

핵심 구현 흐름

2. 방 생성/참여 흐름

방 생성:

1. POST /rooms로 방 생성
2. 자동으로 RoomMember 생성 (방장)
3. 기본 카테고리 자동 생성
4. 초대코드 생성 및 만료시간 설정

방 참여:

1. POST /rooms/join으로 초대코드 검증
2. RoomMember 생성 (일반 멤버)
3. 랜덤 닉네임 및 프로필 이미지 할당

핵심 구현 흐름

3. 집안일 관리 흐름

1. GET /chores로 카테고리 목록 조회
2. 카테고리 선택 후 담당자 및 날짜 설정
3. POST /chores/schedules로 일정 생성
4. 실시간 알림 전송 (집안일 배정)
5. 완료 시 PATCH /chores/schedules/:id/complete

핵심 구현 흐름

4. 예약 시스템 흐름

일반 예약:

1. 요일 + 시간 설정
2. 시간 겹침 검증
3. 바로 승인 상태로 생성

방문객 예약:

1. 특정 날짜 + 시간 설정
2. pending 상태로 생성
3. 다른 멤버들에게 승인 요청 알림
4. 과반수 승인 시 approved 상태로 변경

핵심 구현 흐름

5. 실시간 채팅 흐름

1. Socket.IO 연결

2. join-room 이벤트로 방 참여

3. message 이벤트로 메시지 전송

4. 서버에서 DB 저장 후 방의 모든 사용자에게 브로드캐스트

5. 연결 해제 시 정리 작업

핵심 구현 흐름

6. 알림 시스템 흐름

1. 특정 이벤트 발생 (일정 생성, 예약 승인 등)
2. notificationService에서 알림 데이터 생성
3. Socket.IO를 통한 실시간 알림 전송
4. 모바일 앱에서 알림 표시
5. 사용자 확인 시 읽음 처리

04

효과, 한계 및 확장

문제 상황과 해결 방법, 기대효과,
확장성, 한계점, 회고

04-1 PROBLEM

개발 중 어려움

개발하며 마주친 벽들

1 기술적 진입장벽

Flutter & Node.js 생소함

2 기능 통합 충돌

API 응답 불일치 / Git 충돌



3 소셜 로그인 문제

OAuth 인증 실패 → 방향 전환

4 일정 부족 및 리팩토링

설계의 미흡함 → 중간 수정 반복

시행착오를 겪으며 유연하게 대응하고, 기능 중심으로 개발을 마무리함

04-2 SOLUTION

우리가 해결한 방식

주요 문제 해결 과정



1

기술적 이해도를 빠르게 끌어올림

- 문서와 레퍼런스 학습
- 소규모 단위로 기능 구현
- 공유 및 코드 리뷰

2

기능 통합 문제를 명세서로 해결

- API 응답 규칙 수립
- 에러 처리 방식 통일
- 깃 브랜치 전략 이용

3

Oauth 실패, 전략적 전환

- 임시 인증 구조를 확보하여 다음 단계로 진입

4

일정 압박 → 기능 우선순위 조정

- 사용자 필수 흐름 위주의 개발 범위 재조정
- 회의로 우선순위 설정

04-3 expected

기대효과 및 활용 가능성

방토리가 바꾸는 공동생활 풍경

단순한 앱이 아니라, 함께 사는 공간을 위한 '생활 플랫폼'이 목표

01 룸메이트 갈등 완화	02 커뮤니케이션 공간 제공	03 실질적인 공동생활 관리	04 확장 가능성
세탁기, 욕실 사용 등 일상적인 충돌을 사전 예약 시스템으로 예방	공용 채팅 기능 → 방 안의 소통 활성화	방 생성/초대, 할 일 관리, 체크리스트 등 공동 주거 관리 도구로 활용 가능	캘린더 연동, 미니게임 도입, 쪽지 기능 등 추가 예정
일정 중복 방지, 시각화된 예약 정보로 생활 스트레스 감소	방문자 등록 및 승인 기능으로 외부인 출입 신뢰도 향상	기숙사, 쉐어하우스, 자취방 등 다양한 환경에 적용 가능	관리자 페이지, 통계 기능 등을 통해 확장 기반 마련

04-4 LIMITATION

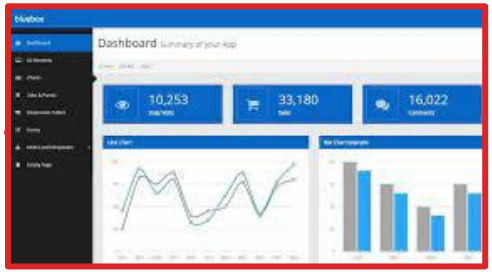
한계점과 보완 계획

한계점과 보완 계획

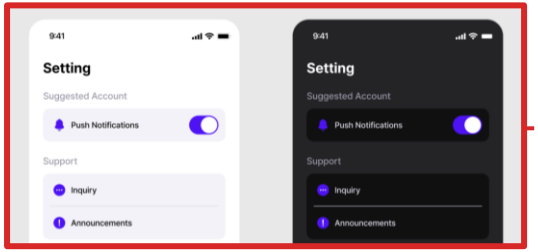
OAuth
소셜 로그인 지원 예정



Admin page
관리자 페이지 구현 예정

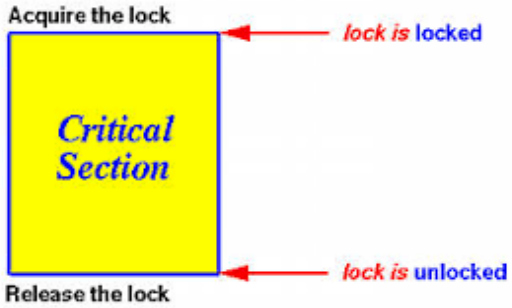


Dark mode
다크모드 기능



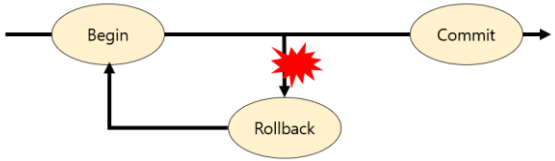
Design pattern
디자인 패턴을 활용

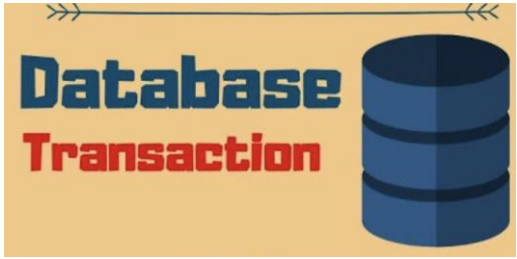




락과 트랜잭션

현재에 안주하지 않고,
피드백을 바탕으로
계속 발전시킬 예정





LOCK

<충돌 방지 필요 구간>

- **방 참여:** 같은 코드로 동시 참여 가능
- **방장 위임**: 위임 중 다른 작업 동시 발생 가능
- **예약 등록**: 동일 시간대 예약 동시 시도 가능



TRANSACTION

<데이터 정합성 및 안정성 확보>

- 핵심 기능들이 여러 단계의 데이터 변경을 포함
- 중간에 일부만 반영될 경우 데이터 불일치나 논리 오류가 발생할 수 있음